AI Assisted Coding Lab ass-13.4

NAME : G. SANJAY

2303A510B9

B- 13

Task 1: Refactoring Data Transformation Logic

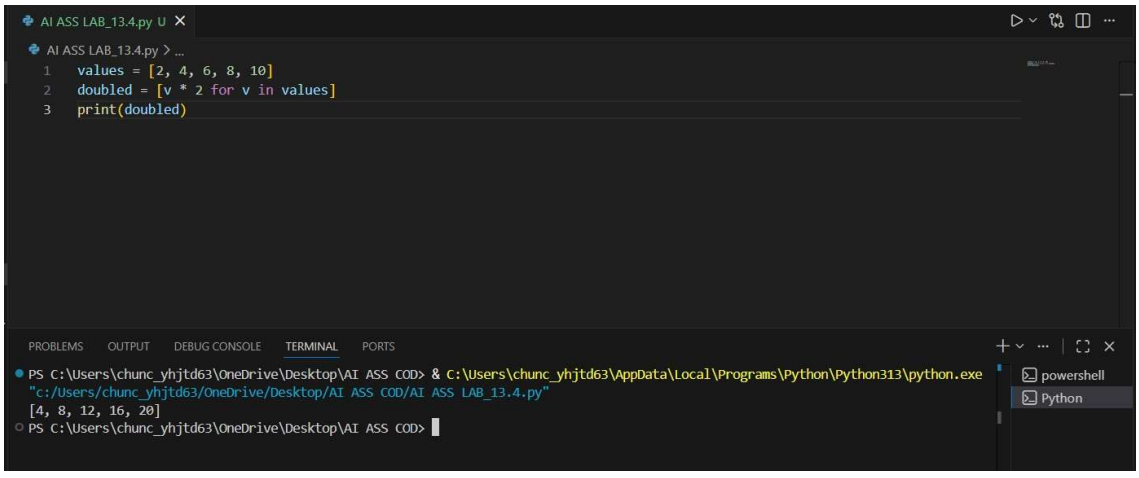
Task Description

- **Review the legacy code that computes transformed values using an explicit loop.**
- **Use an AI tool to suggest a more Pythonic refactoring approach.**
- **Refactor the code using list comprehensions or helper functions while preserving the output.**

PROMPT:

Refactor the following Python code to make it shorter and more Pythonic using list comprehension while keeping the same output.

```
values = [2, 4, 6, 8, 10]
doubled = [] for v in
values:
doubled.append(v * 2)
print(doubled)
```



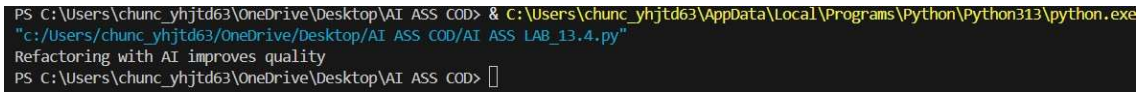
Task 2: Improving Text Processing Code Readability

- **Analyze the legacy code that constructs a sentence using repeated string concatenation.**
- **Ask AI to suggest a more efficient and readable approach.**
- **Refactor the code accordingly while keeping the final output unchanged.**

PROMPT:

Refactor the following Python code to improve readability and efficiency by avoiding repeated string concatenation. Keep the same output.

OUTPUT:



Task 3: Safer Access to Configuration Data

- **Review the legacy code that manually checks for dictionary keys.**
- **Use AI suggestions to refactor the code using safer dictionary access methods.**
- **Ensure the behavior remains the same for missing keys.**

PROMPT:

Refactor the following Python code to use a safer dictionary access method instead of manually checking keys. Keep the same output if the key is missing.

```
config = {"host": "localhost", "port": 8080} if
"timeout" in config:
   print(config["timeout"]) else:
print("Default timeout used")
```

```
config = {"host": "localhost", "port": 8080}
print(config.get("timeout", "Default timeout used"))
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe
"c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
Default timeout used
```

Task 4: Refactoring Conditional Logic for Scalability

Examine the multiple if–elif conditions used to determine operations.

Ask AI to suggest a cleaner, scalable alternative.

Refactor the logic using mapping techniques while preserving functionality.

PROMPT:

Refactor the following Python code to replace multiple if–elif conditions with a cleaner and scalable mapping approach while keeping the same output.

```
action = "divide"
x, y = 10, 2

operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b,
}

result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe
"c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
5.0
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>
```

Task 5: Simplifying Search Logic in Collections

Identify the explicit loop used for searching an item.

Use AI assistance to refactor the logic into a more concise and readable form.

Maintain the same output behavior.

PROMPT:

Refactor the following Python code to simplify the search logic in a collection using a more concise Python approach while keeping the same output.

OUTPUT: